

SET - I

1. University Academic Database

Question

Design and implement a University Academic Database for student-course management. Normalize up to 3NF, define constraints, insert sample data, and develop PL/SQL modules for result processing and reporting.

Aim

To design and implement a University Academic Database with normalization, constraints, and result processing.

Algorithm

1. Create STUDENT, COURSE, and ENROLL tables.
 2. Apply primary key and foreign key constraints.
 3. Insert sample data.
 4. Normalize database up to 3NF.
 5. Generate result reports using SQL queries.
-

MySQL Queries

```
CREATE DATABASE UniversityDB;
```

```
USE UniversityDB;
```

```
CREATE TABLE Student(  
    SID INT PRIMARY KEY,  
    NAME VARCHAR(50),  
    DEPT VARCHAR(30)  
);
```

```
CREATE TABLE Course(  
    CID INT PRIMARY KEY,  
    NAME VARCHAR(50),  
    DEPT VARCHAR(30)
```

```
CID INT PRIMARY KEY,  
CNAME VARCHAR(50),  
CREDITS INT  
);
```

```
CREATE TABLE Enroll(  
    SID INT,  
    CID INT,  
    MARKS INT,  
    PRIMARY KEY(SID,CID),  
    FOREIGN KEY(SID) REFERENCES Student(SID),  
    FOREIGN KEY(CID) REFERENCES Course(CID)  
);
```

```
INSERT INTO Student VALUES  
(1,'Arun','CSE'),  
(2,'Priya','IT');
```

```
INSERT INTO Course VALUES  
(101,'DBMS',4),  
(102,'Java',3);
```

```
INSERT INTO Enroll VALUES  
(1,101,85),  
(1,102,90),  
(2,101,75);
```

```
SELECT S.NAME,C.CNAME,E.MARKS
```

FROM Student S

JOIN Enroll E ON S.SID=E.SID

JOIN Course C ON E.CID=C.CID;

Result

Thus the University Academic Database was implemented successfully.

2. Student and Marks Queries

Aim

To perform SQL queries using joins and subqueries on STUDENT and MARKS tables.

Algorithm

1. Create STUDENT and MARKS tables.
 2. Insert sample data.
 3. Execute join and subquery operations.
-

MySQL Queries

```
CREATE TABLE Student(  
    SID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    DEPT VARCHAR(20)  
);
```

```
CREATE TABLE Marks(  
    SID INT,  
    SUBJECT VARCHAR(30),  
    SCORE INT  
);
```

```
INSERT INTO Student VALUES  
(1,'Arun','CSE'),  
(2,'Priya','IT');
```

```
INSERT INTO Marks VALUES  
(1,'DBMS',85),
```

```
(1,'OS',90),  
(2,'DBMS',70);
```

-- Department-wise toppers

```
SELECT S.DEPT,S.NAME,MAX(M.SCORE) AS TOP_SCORE  
FROM Student S  
JOIN Marks M ON S.SID=M.SID  
GROUP BY S.DEPT;
```

-- Subject averages

```
SELECT SUBJECT,AVG(SCORE) AS AVG_MARK  
FROM Marks  
GROUP BY SUBJECT;
```

-- Students above overall average

```
SELECT SID,SCORE  
FROM Marks  
WHERE SCORE >  
(SELECT AVG(SCORE) FROM Marks);
```

Result

Thus the required SQL queries were executed successfully.

3. Hospital Management System

Aim

To develop a Hospital Management System with triggers and billing procedures.

Algorithm

1. Create Patient, Doctor, and Appointment tables.
 2. Create audit trigger.
 3. Generate analytical reports.
-

MySQL Queries

```
CREATE TABLE Doctor(  
    DID INT PRIMARY KEY,  
    DNAME VARCHAR(30)  
);
```

```
CREATE TABLE Patient(  
    PID INT PRIMARY KEY,  
    PNAME VARCHAR(30)  
);
```

```
CREATE TABLE Appointment(  
    AID INT PRIMARY KEY,  
    PID INT,  
    DID INT,  
    FEES INT,  
    FOREIGN KEY(PID) REFERENCES Patient(PID),  
    FOREIGN KEY(DID) REFERENCES Doctor(DID)  
);
```

```
CREATE TABLE AuditLog(  
    ID INT AUTO_INCREMENT PRIMARY KEY,  
    MESSAGE VARCHAR(100)  
);
```

```
DELIMITER //  
  
CREATE TRIGGER appointment_log  
AFTER INSERT ON Appointment  
FOR EACH ROW  
BEGIN  
    INSERT INTO AuditLog(MESSAGE)  
    VALUES('Appointment Added');  
END//  
DELIMITER ;
```

```
SELECT * FROM AuditLog;
```

Result

Thus the Hospital Management System was implemented successfully.

4. Employee and Department Queries

Aim

To perform correlated subqueries and salary analysis.

MySQL Queries

```
CREATE TABLE Dept(  
    DID INT PRIMARY KEY,  
    DNAME VARCHAR(30)  
);
```

```
CREATE TABLE Emp(  
    EID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    DID INT,  
    SALARY INT,  
    FOREIGN KEY(DID) REFERENCES Dept(DID)  
);
```

```
INSERT INTO Dept VALUES  
(1,'CSE'),  
(2,'IT');
```

```
INSERT INTO Emp VALUES  
(101,'Arun',1,50000),  
(102,'Priya',2,70000),  
(103,'Kiran',1,60000);
```

-- Highest paid employee


```
SELECT DNAME,MAX(SALARY)
FROM Emp E
JOIN Dept D ON E.DID=D.DID
GROUP BY DNAME;
```

-- Departments above threshold

```
SELECT DID,AVG(SALARY)
FROM Emp
GROUP BY DID
HAVING AVG(SALARY)>55000;
```

-- Correlated subquery

```
SELECT NAME,SALARY
FROM Emp E1
WHERE SALARY=
(
SELECT MAX(SALARY)
FROM Emp E2
WHERE E1.DID=E2.DID
);
```

Result

Thus the employee salary analysis was completed successfully.

5. Banking System

Aim

To implement a Banking System using TCL commands and fund transfer procedures.

Algorithm

1. Create Account table.
 2. Insert balances.
 3. Perform transaction using COMMIT, ROLLBACK, and SAVEPOINT.
-

MySQL Queries

```
CREATE TABLE Account(  
    AID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    BALANCE INT  
);
```

```
INSERT INTO Account VALUES  
(1,'Arun',10000),  
(2,'Priya',8000);
```

```
START TRANSACTION;
```

```
UPDATE Account  
SET BALANCE=BALANCE-2000  
WHERE AID=1;
```

```
SAVEPOINT transfer_point;
```

UPDATE Account

SET BALANCE=BALANCE+2000

WHERE AID=2;

COMMIT;

SELECT * FROM Account;

Result

Thus the Banking System transaction was executed successfully.

6. Sales Queries

Aim

To compute monthly revenue and top customers using aggregation.

MySQL Queries

```
CREATE TABLE Sales(
```

```
    PID INT,
```

```
    CID INT,
```

```
    AMOUNT INT,
```

```
    SDATE DATE
```

```
);
```

```
INSERT INTO Sales VALUES
```

```
(101,1,5000,'2026-01-10'),
```

```
(102,2,7000,'2026-01-15'),
```

```
(103,1,4000,'2026-02-01');
```

```
-- Monthly Revenue
```

```
SELECT MONTH(SDATE) AS MONTH,
```

```
SUM(AMOUNT) AS REVENUE
```

```
FROM Sales
```

```
GROUP BY MONTH(SDATE);
```

```
-- Top 5 Customers
```

```
SELECT CID,SUM(AMOUNT) AS TOTAL
```

```
FROM Sales
```

```
GROUP BY CID
```

```
ORDER BY TOTAL DESC
```

LIMIT 5;

-- Average Sales

SELECT AVG(AMOUNT)

FROM Sales;

Result

Thus the sales queries were executed successfully.

7. Library Management System

Aim

To implement fine calculation and overdue triggers in Library Management System.

MySQL Queries

CREATE TABLE Library(

BID INT PRIMARY KEY,

BOOKNAME VARCHAR(50),

DUE_DAYS INT

);

DELIMITER //

CREATE FUNCTION Fine(days INT)

RETURNS INT

DETERMINISTIC

BEGIN

RETURN days*5;

END//

DELIMITER ;

SELECT Fine(4) AS FINE_AMOUNT;

Result

Thus the Library Management System was implemented successfully.

8. Nested Query Operations

Aim

To perform nested queries using IN, EXISTS, ANY, and ALL.

MySQL Queries

```
CREATE TABLE Orders(
```

```
    OID INT PRIMARY KEY,
```

```
    CID INT,
```

```
    AMOUNT INT
```

```
);
```

```
INSERT INTO Orders VALUES
```

```
(1,101,5000),
```

```
(2,102,7000),
```

```
(3,101,3000);
```

```
-- IN Query
```

```
SELECT *
```

```
FROM Orders
```

```
WHERE CID IN
```

```
(SELECT CID FROM Orders WHERE AMOUNT>5000);
```

```
-- EXISTS Query
```

```
SELECT *
```

```
FROM Orders O1
```

```
WHERE EXISTS
```

```
(
```

```
SELECT *
```

```
FROM Orders O2
WHERE O1.CID=O2.CID
AND O2.AMOUNT>6000
);
```

```
-- ANY Query
SELECT *
FROM Orders
WHERE AMOUNT > ANY
(SELECT AMOUNT FROM Orders WHERE CID=101);
```

```
-- ALL Query
SELECT *
FROM Orders
WHERE AMOUNT > ALL
(SELECT AMOUNT FROM Orders WHERE CID=101);
```

Result

Thus the nested queries were executed successfully.

9. Online Retail Database

Aim

To implement discount calculation and sales analytics.

MySQL Queries

```
CREATE TABLE Retail(
```

```
    OID INT PRIMARY KEY,
```

```
    PRODUCT VARCHAR(30),
```

```
    PRICE INT
```

```
);
```

```
DELIMITER //
```

```
CREATE FUNCTION Discount(price INT)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
RETURN price-(price*10/100);
```

```
END//
```

```
DELIMITER ;
```

```
INSERT INTO Retail VALUES
```

```
(1,'Laptop',50000);
```

```
SELECT PRODUCT,
```

```
PRICE,
```

```
Discount(PRICE) AS FINAL_PRICE
```

```
FROM Retail;
```

Result

Thus the Online Retail Database was implemented successfully.

10. JOIN Operations

Aim

To demonstrate INNER, LEFT, RIGHT, and FULL JOIN operations.

MySQL Queries

```
CREATE TABLE Student(  
    SID INT PRIMARY KEY,  
    NAME VARCHAR(30)  
);
```

```
CREATE TABLE Course(  
    CID INT PRIMARY KEY,  
    CNAME VARCHAR(30),  
    SID INT  
);
```

-- INNER JOIN

```
SELECT *  
FROM Student  
INNER JOIN Course  
ON Student.SID=Course.SID;
```

-- LEFT JOIN

```
SELECT *  
FROM Student  
LEFT JOIN Course  
ON Student.SID=Course.SID;
```

```
-- RIGHT JOIN  
SELECT *  
FROM Student  
RIGHT JOIN Course  
ON Student.SID=Course.SID;
```

```
-- FULL JOIN Simulation  
SELECT *  
FROM Student  
LEFT JOIN Course  
ON Student.SID=Course.SID
```

UNION

```
SELECT *  
FROM Student  
RIGHT JOIN Course  
ON Student.SID=Course.SID;
```

Result

Thus all JOIN operations were demonstrated successfully.

11. Employee Payroll System

Aim

To implement salary computation and payroll reports.

MySQL Queries

```
CREATE TABLE Payroll(
```

```
    EID INT PRIMARY KEY,
```

```
    ENAME VARCHAR(30),
```

```
    DEPT VARCHAR(30),
```

```
    BASIC INT
```

```
);
```

```
DELIMITER //
```

```
CREATE FUNCTION NetSalary(basic INT)
```

```
RETURNS INT
```

```
DETERMINISTIC
```

```
BEGIN
```

```
RETURN basic + (basic*20/100);
```

```
END//
```

```
DELIMITER ;
```

```
INSERT INTO Payroll VALUES
```

```
(1,'Arun','CSE',50000);
```

```
SELECT ENAME,
```

```
NetSalary(BASIC) AS NETSALARY
```

```
FROM Payroll;
```

Result

Thus the Payroll System was implemented successfully.

12. Account Trigger System

Aim

To prevent negative balance using triggers and maintain transaction history.

MySQL Queries

```
CREATE TABLE Account(
```

```
    AID INT PRIMARY KEY,
```

```
    BALANCE INT
```

```
);
```

```
CREATE TABLE TransactionHistory(
```

```
    TID INT AUTO_INCREMENT PRIMARY KEY,
```

```
    MESSAGE VARCHAR(100)
```

```
);
```

```
DELIMITER //
```

```
CREATE TRIGGER balance_check
```

```
BEFORE UPDATE ON Account
```

```
FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.BALANCE < 0 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Negative Balance Not Allowed';
```

```
END IF;
```

```
END//
```

```
DELIMITER ;
```

```
INSERT INTO Account VALUES
```

(1,5000);

UPDATE Account

SET BALANCE=3000

WHERE AID=1;

Result

Thus the trigger execution was demonstrated successfully.

13. Airline Reservation System

Question

Design an Airline Reservation System. Implement constraints, triggers for seat allocation, and queries for booking analysis.

Aim

To design and implement an Airline Reservation System with seat allocation constraints and booking analysis.

Algorithm

1. Create Flight and Reservation tables.
 2. Apply primary key and foreign key constraints.
 3. Create trigger for seat validation.
 4. Insert reservation records.
 5. Generate booking analysis reports.
-

MySQL Queries

```
CREATE DATABASE AirlineDB;  
USE AirlineDB;
```

```
CREATE TABLE Flight(  
    FID INT PRIMARY KEY,  
    FNAME VARCHAR(50),  
    TOTAL_SEATS INT  
);
```

```
CREATE TABLE Reservation(  
    RID INT PRIMARY KEY,  
    FID INT,  
    PASSENGER_NAME VARCHAR(50),  
    SEATS_BOOKED INT,  
    BOOKING_DATE DATE,  
    FOREIGN KEY(FID) REFERENCES Flight(FID)  
);
```

```
INSERT INTO Flight VALUES
```

```
(1,'AirIndia',100),
```

```
(2,'Indigo',80);
```

```
DELIMITER //
```

```
CREATE TRIGGER seat_allocation
```

```
BEFORE INSERT ON Reservation
```

```
FOR EACH ROW
```

```
BEGIN
```

```
DECLARE booked INT;
```

```
SELECT IFNULL(SUM(SEATS_BOOKED),0)
```

```
INTO booked
```

```
FROM Reservation
```

```
WHERE FID=NEW.FID;
```

```
IF booked + NEW.SEATS_BOOKED >
```

```
  (SELECT TOTAL_SEATS FROM Flight WHERE FID=NEW.FID)
```

```
THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT='Seats Not Available';
```

```
END IF;
```

```
END//
```

```
DELIMITER ;
```

```
INSERT INTO Reservation VALUES
```

```
(101,1,'Arun',5,'2026-05-10');
```

```
-- Booking Analysis
```

```
SELECT FID,
```

```
SUM(SEATS_BOOKED) AS TOTAL_BOOKINGS
```

```
FROM Reservation
```

```
GROUP BY FID;
```

Result

Thus the Airline Reservation System was implemented successfully.

14. Customer Table Optimization

Question

Given CUSTOMER table with large data:

Create views for restricted access, implement indexing, and analyze query performance improvement.

Aim

To implement views and indexing for improving query performance.

Algorithm

1. Create CUSTOMER table.
 2. Insert records.
 3. Create restricted-access view.
 4. Create indexes on frequently queried columns.
 5. Compare query performance.
-

MySQL Queries

```
CREATE TABLE Customer(  
  CID INT PRIMARY KEY,  
  NAME VARCHAR(30),  
  CITY VARCHAR(30),  
  PHONE VARCHAR(15)  
);
```

```
INSERT INTO Customer VALUES  
(1,'Arun','Chennai','9876543210'),  
(2,'Priya','Madurai','9876501234');
```

-- Create View

```
CREATE VIEW CustomerView AS  
SELECT CID,NAME,CITY  
FROM Customer;
```

```
SELECT * FROM CustomerView;
```

```
-- Create Index  
CREATE INDEX idx_city  
ON Customer(CITY);
```

```
EXPLAIN  
SELECT * FROM Customer  
WHERE CITY='Chennai';
```

Performance Improvement

- Restricted access provided through views.
 - Faster searching using indexes.
 - Reduced table scanning.
-

Result

Thus views and indexing were implemented successfully.

15. Inventory Management System

Question

Design an Inventory Management System. Implement triggers for automatic stock updates and generate reorder-level reports.

Aim

To implement automatic stock update triggers and reorder-level analysis.

Algorithm

1. Create PRODUCT and SALES tables.
 2. Insert records.
 3. Create trigger for stock reduction after sales.
 4. Generate reorder reports.
-

MySQL Queries

```
CREATE TABLE Product(  
    PID INT PRIMARY KEY,  
    PNAME VARCHAR(30),  
    STOCK INT,  
    REORDER_LEVEL INT  
);
```

```
CREATE TABLE Sales(  
    SID INT PRIMARY KEY,  
    PID INT,  
    QUANTITY INT,  
    FOREIGN KEY(PID) REFERENCES Product(PID)  
);
```

```
INSERT INTO Product VALUES  
(1,'Laptop',50,10),  
(2,'Mouse',100,20);
```

```
DELIMITER //  
CREATE TRIGGER stock_update
```

```
AFTER INSERT ON Sales
FOR EACH ROW
BEGIN
UPDATE Product
SET STOCK = STOCK - NEW.QUANTITY
WHERE PID = NEW.PID;
END//
DELIMITER ;
```

```
INSERT INTO Sales VALUES
(101,1,5);
```

```
-- Reorder Level Report
SELECT *
FROM Product
WHERE STOCK < REORDER_LEVEL;
```

Result

Thus the Inventory Management System was implemented successfully.

16. TCL and DCL Commands

Question

Demonstrate TCL and DCL commands:

Perform a transaction using COMMIT, ROLLBACK, SAVEPOINT and apply GRANT/REVOKE for different users with explanation.

Aim

To demonstrate TCL and DCL commands in MySQL.

Algorithm

1. Create ACCOUNT table.
 2. Perform transactions using TCL commands.
 3. Apply GRANT and REVOKE permissions.
-

MySQL Queries

```
CREATE TABLE Account(  
    AID INT PRIMARY KEY,  
    NAME VARCHAR(30),  
    BALANCE INT  
);
```

```
INSERT INTO Account VALUES  
(1,'Arun',10000),  
(2,'Priya',8000);
```

```
-- TCL Commands  
START TRANSACTION;
```

```
UPDATE Account  
SET BALANCE=BALANCE-2000  
WHERE AID=1;
```

```
SAVEPOINT sp1;
```

```
UPDATE Account
```

```
SET BALANCE=BALANCE+2000  
WHERE AID=2;
```

```
COMMIT;
```

```
-- ROLLBACK Example  
ROLLBACK TO sp1;
```

```
-- DCL Commands  
GRANT SELECT,INSERT  
ON Account  
TO 'user1'@'localhost';
```

```
REVOKE INSERT  
ON Account  
FROM 'user1'@'localhost';
```

Explanation

- COMMIT permanently saves transaction.
- ROLLBACK undoes transaction changes.
- SAVEPOINT creates checkpoints.
- GRANT provides permissions.
- REVOKE removes permissions.

Result

Thus TCL and DCL commands were demonstrated successfully.

17. Product Analysis and Indexing

Question

Given PRODUCT(PID, NAME, CATEGORY, PRICE, STOCK):

Perform queries to identify top-selling categories, low stock products, indexing, and performance analysis.

Aim

To analyze product sales and optimize queries using indexing.

Algorithm

1. Create PRODUCT table.
 2. Insert sample records.
 3. Execute analytical queries.
 4. Create indexes.
 5. Compare performance.
-

MySQL Queries

```
CREATE TABLE Product(  
  PID INT PRIMARY KEY,  
  NAME VARCHAR(30),  
  CATEGORY VARCHAR(30),  
  PRICE INT,  
  STOCK INT  
);
```

```
INSERT INTO Product VALUES  
(1,'Laptop','Electronics',50000,10),  
(2,'Mouse','Electronics',500,100),  
(3,'Chair','Furniture',3000,5);
```

```
-- Top Selling Categories  
SELECT CATEGORY,  
COUNT(*) AS TOTAL_PRODUCTS  
FROM Product
```

```
GROUP BY CATEGORY
ORDER BY TOTAL_PRODUCTS DESC;
```

```
-- Low Stock Products
SELECT *
FROM Product
WHERE STOCK < 10;
```

```
-- Create Indexes
CREATE INDEX idx_category
ON Product(CATEGORY);
```

```
CREATE INDEX idx_stock
ON Product(STOCK);
```

```
EXPLAIN
SELECT * FROM Product
WHERE CATEGORY='Electronics';
```

Performance Improvement

- Faster category-based retrieval.
- Improved stock filtering.
- Reduced execution time.

Result

Thus the product analysis and indexing were implemented successfully.

18. NoSQL Document-Based Schema

Question

Design a NoSQL document-based schema (JSON) for customer orders. Perform retrieval operations and compare with relational schema.

Aim

To design a NoSQL JSON schema and compare it with relational schema.

Algorithm

1. Design JSON document structure.
 2. Store customer order details.
 3. Retrieve records.
 4. Compare NoSQL and relational database approaches.
-

JSON Schema

```
{
  "customer_id": 101,
  "customer_name": "Arun",
  "orders": [
    {
      "order_id": 1,
      "product": "Laptop",
      "amount": 50000
    },
    {
      "order_id": 2,
      "product": "Mouse",
      "amount": 500
    }
  ]
}
```

Retrieval Example

```
db.orders.find(  
  {customer_id:101}  
)
```

Comparison

Relational Schema	NoSQL Schema
Uses tables	Uses JSON documents
Fixed schema	Flexible schema
Complex joins needed	Nested data supported
Best for structured data	Best for semi-structured data

Result

Thus the NoSQL document schema was designed and retrieval operations were performed successfully.

19. Firebase Real-Time Task Management Application

Question

Develop a Firebase Real-time Database application for task management. Demonstrate real-time updates and analyze consistency issues.

Aim

To develop a Firebase-based task management application with real-time synchronization.

Algorithm

1. Create Firebase project.
 2. Enable Firebase Realtime Database.
 3. Connect application to Firebase.
 4. Add and retrieve tasks in real time.
 5. Analyze consistency.
-

HTML + Firebase Code

```
<!DOCTYPE html>
<html>
<head>
  <title>Task Manager</title>
</head>
<body>

  <input type="text" id="task">
  <button onclick="addTask()">Add Task</button>

  <ul id="tasks"></ul>

  <script src="https://www.gstatic.com/firebasejs/8.6.8/firebase-app.js"></script>
  <script src="https://www.gstatic.com/firebasejs/8.6.8/firebase-database.js"></script>

  <script>

  var firebaseConfig = {
```

```
    apiKey: "YOUR_API_KEY",
    authDomain: "taskapp.firebaseio.com",
    databaseURL: "https://taskapp.firebaseio.com",
    projectId: "taskapp"
  };
```

```
firebase.initializeApp(firebaseConfig);
```

```
var db = firebase.database();
```

```
function addTask(){
```

```
    var task =
    document.getElementById("task").value;
```

```
    db.ref("tasks").push({
        taskname: task
    });
}
```

```
db.ref("tasks").on("child_added",
function(data){
```

```
    var li=document.createElement("li");
```

```
    li.innerText=data.val().taskname;
```

```
    document.getElementById("tasks")
    .appendChild(li);
});
```

```
</script>
```

```
</body>
```

```
</html>
```

Consistency Issues

- Simultaneous updates may overwrite data.
- Firebase synchronization resolves conflicts automatically.

- Real-time updates improve collaboration.
-

Result

Thus the Firebase Real-time Database application was developed successfully.

20. Retail Dataset Analytical Queries

Question

Given a retail dataset:

Perform analytical queries to compute category-wise sales, identify trends, and interpret results for decision-making.

Aim

To analyze retail sales trends and generate business insights.

Algorithm

1. Create RETAIL table.
 2. Insert sales records.
 3. Perform category-wise analysis.
 4. Identify trends using aggregation queries.
 5. Interpret analytical results.
-

MySQL Queries

```
CREATE TABLE Retail(  
  RID INT PRIMARY KEY,  
  CATEGORY VARCHAR(30),  
  PRODUCT VARCHAR(30),  
  AMOUNT INT,  
  SALE_DATE DATE  
);
```

```
INSERT INTO Retail VALUES  
(1,'Electronics','Laptop',50000,'2026-01-10'),  
(2,'Electronics','Mobile',30000,'2026-01-15'),  
(3,'Furniture','Chair',10000,'2026-02-01');
```

```
-- Category-wise Sales  
SELECT CATEGORY,  
SUM(AMOUNT) AS TOTAL_SALES  
FROM Retail
```


GROUP BY CATEGORY;

-- Monthly Trends

```
SELECT MONTH(SALE_DATE) AS MONTH,  
SUM(AMOUNT) AS MONTHLY_REVENUE  
FROM Retail  
GROUP BY MONTH(SALE_DATE);
```

-- Best Selling Products

```
SELECT PRODUCT,  
SUM(AMOUNT) AS SALES  
FROM Retail  
GROUP BY PRODUCT  
ORDER BY SALES DESC;
```

Interpretation

- **Electronics category has highest sales.**
- **Monthly revenue helps identify seasonal demand.**
- **Best-selling products support business decisions.**

Result

Thus analytical queries were performed successfully and trends were identified.